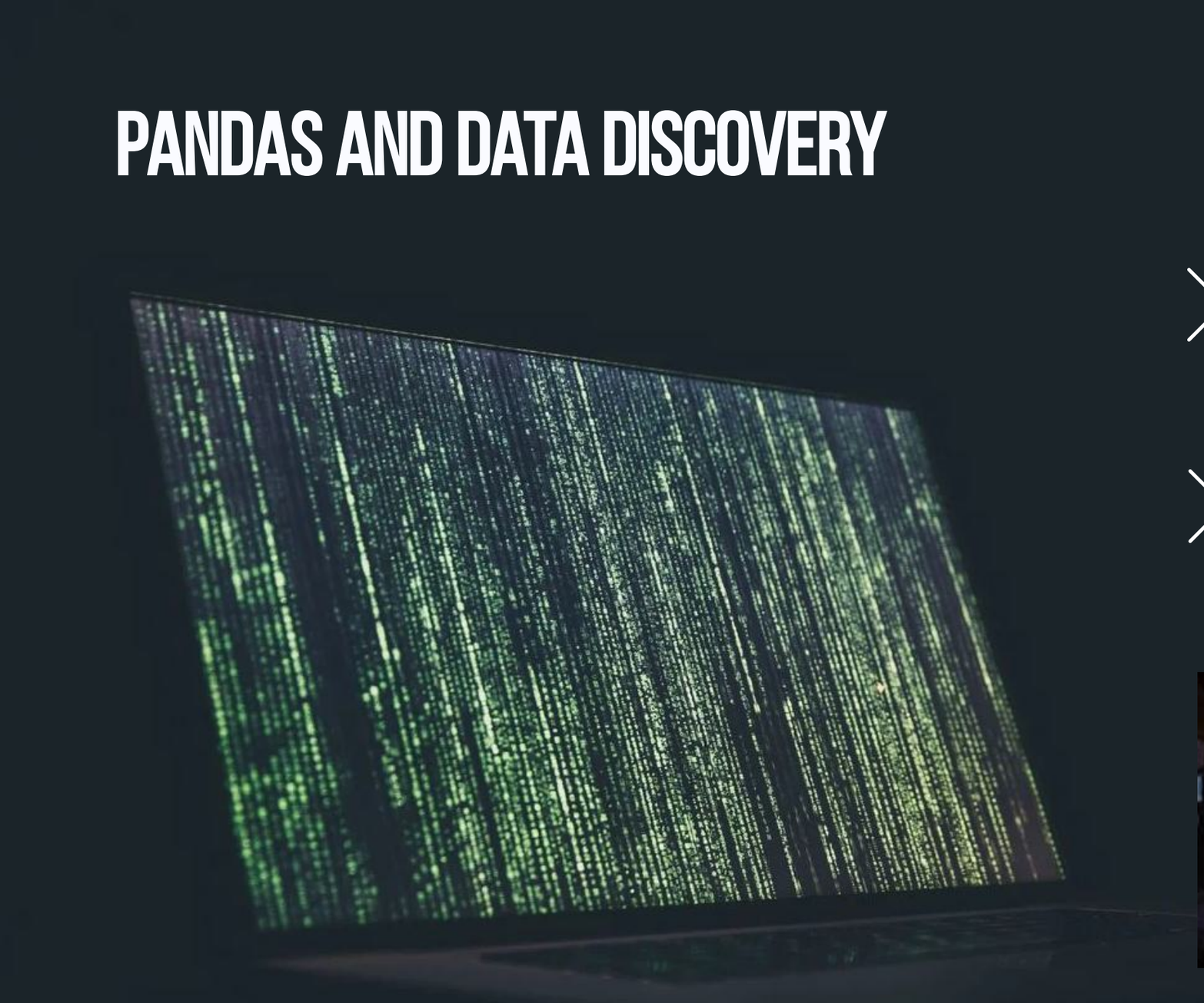
# PANDAS AND DATA DISCOVERY



- MANIPULATING DATAFRAMES AND STATISTICAL ANALYSIS
- PRACTICAL 8




David Topping: david.topping@manchester.ac.uk

'Vanilla' Python

Data types
Basic operations

Loops
Conditionals

Pandas and data discovery

Geospatial analysis

Statistical Analysis

Cluster Analysis

1 2 3 4 5 6 7 8 9

The Numpy module

Conway's Game Of Life

Differential Equations

Modules and Functions

Creating a notebook assessment
select topic from weeks 4 - 9

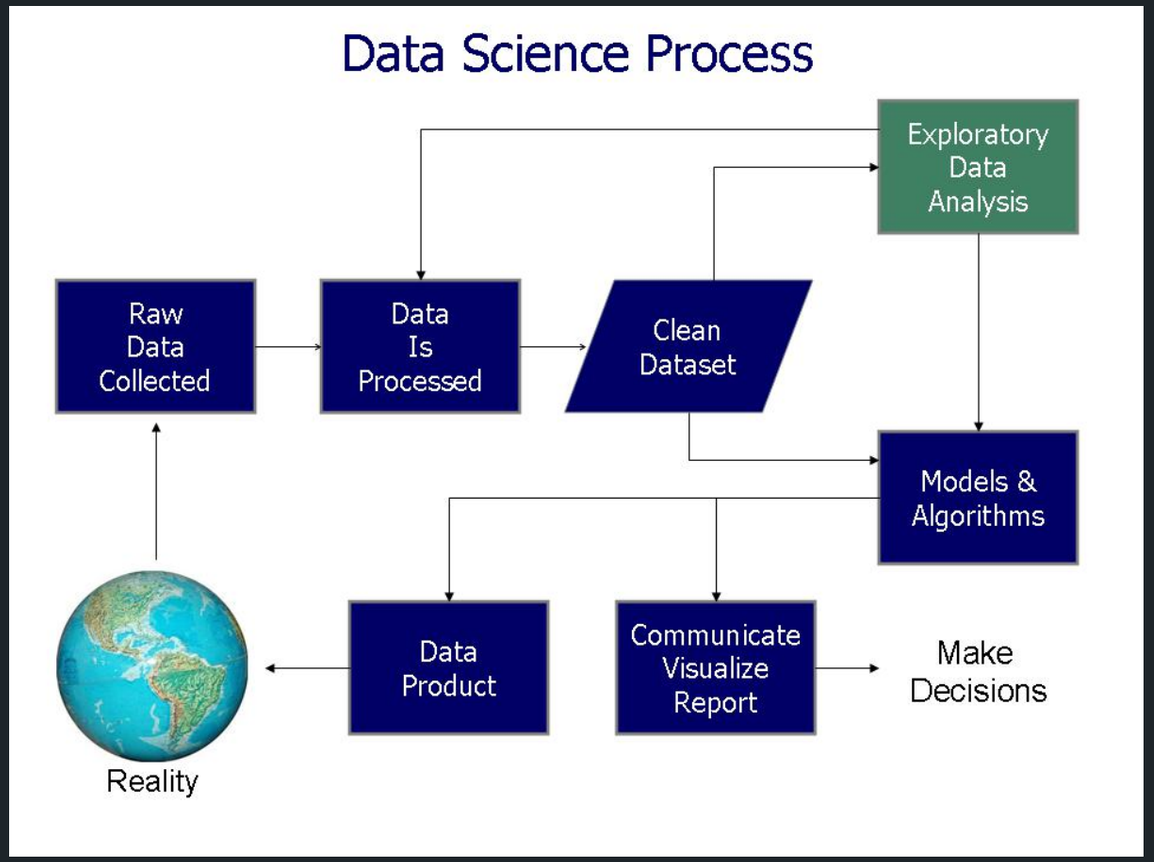
10 11 12

# COURSE LAYOUT >>>>

LECTURE + PRACTICAL. SOLUTIONS ARE GIVEN

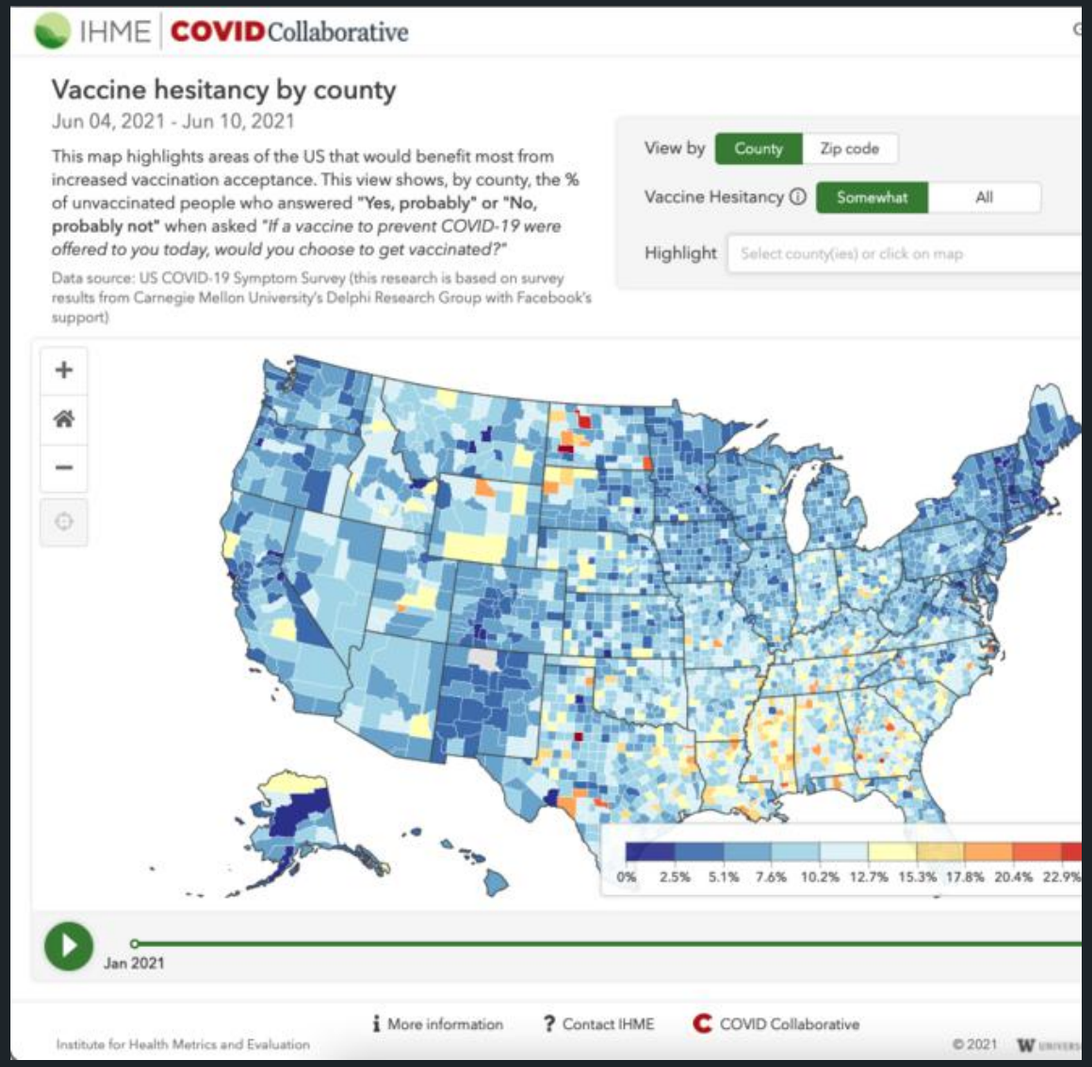
# RECAP

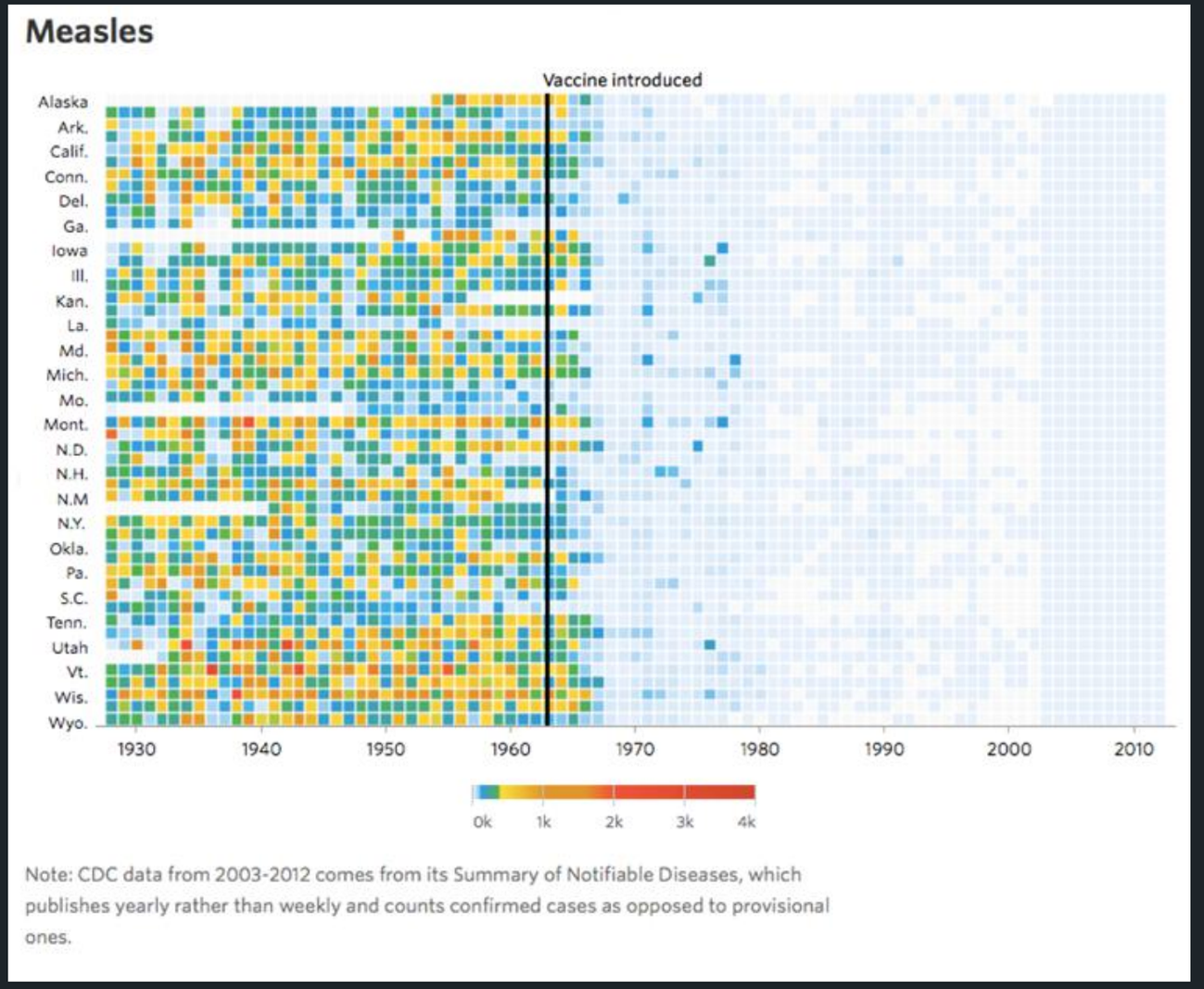
- Pandas allows us to store information in a dataframe
- Pandas can deal with various data types
- We can access entries in that dataframe according to certain criteria
- Dataframes have internal functions we can use to our advantage



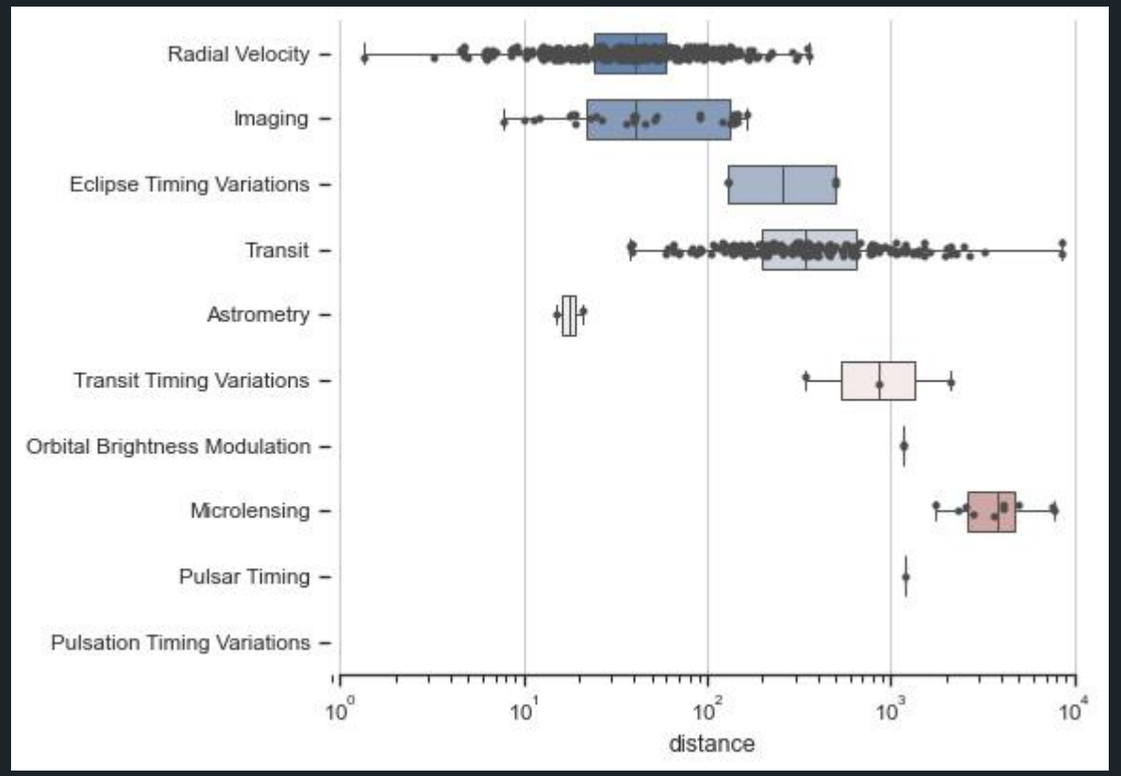

What if we have data we want to perform statistical analysis on the data?

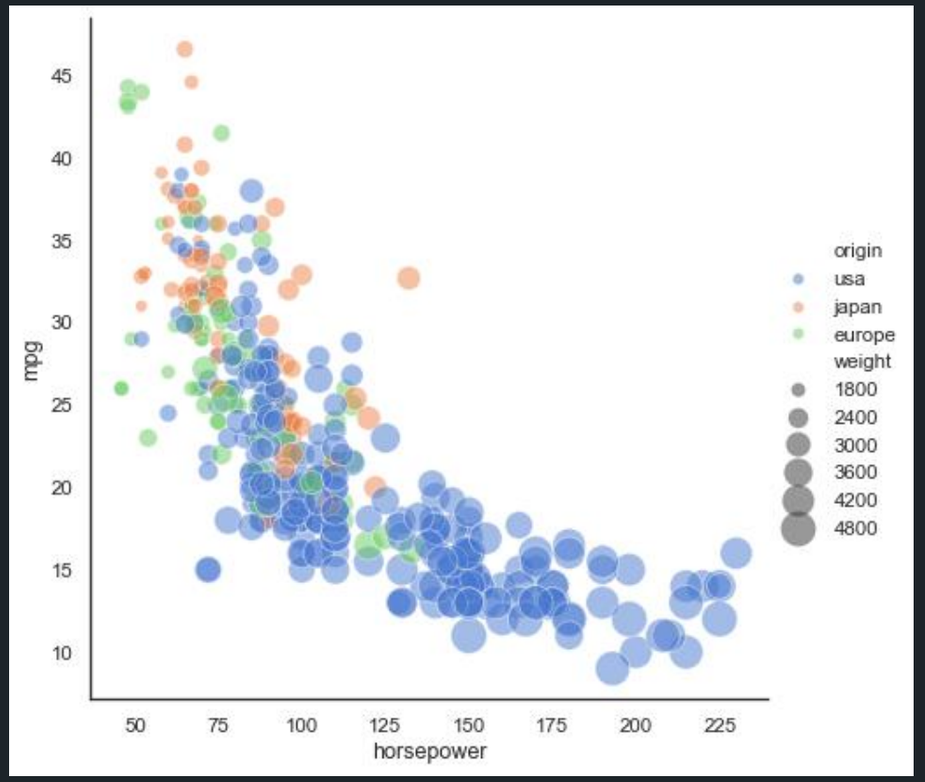
# STATISTICAL VISUALISATIONS CAN TELL A STORY



# STATISTICAL VISUALISATIONS CAN TELL A STORY



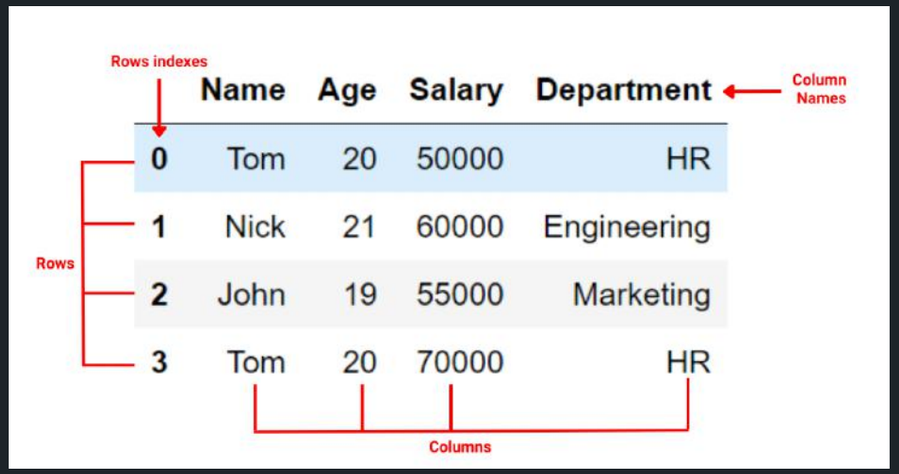



What if we have data we want to perform statistical analysis on the data?

Setting the index of a dataframe
- What is an index?

Merging two dataframes
- Why do we want to do this?

Performing statistical analysis
- Understanding our data!
> Correlation coefficients
> Pair plots
> Box-plots




In [17]:
```
data=pd.DataFrame(
    {"x":range(8),"y":range(8,0,-1),
     "a":["one","one","one","one","two",
          "two","two","two"],
     "b":[0,1,2,3,0,1,2,3]})
data
```

Out[17]:

| | x | y | a | b |
|---|---|---|---|---|
| 0 | 0 | 8 | one | 0 |
| 1 | 1 | 7 | one | 1 |
| 2 | 2 | 6 | one | 2 |
| 3 | 3 | 5 | one | 3 |
| 4 | 4 | 4 | two | 0 |
| 5 | 5 | 3 | two | 1 |
| 6 | 6 | 2 | two | 2 |
| 7 | 7 | 1 | two | 3 |

# WHAT IS AN INDEX?

Each column has a name associated with it, also known as a label.
We can access entries in specific columns:

```
Latitude = data['Latitude'].values # Extract the 'Latitude' column and th
Latitude #This will just print a preview of our array to the screen.
```

```
array([ -5.05 , -16.355,  48.292, ...,  38.654,  43.42 ,  -4.889])
```

```
Longitude = data['Longitude'].values # We can do the same with Longitude
Longitude
```

```
array([ 151.33 ,  -70.903,  153.25 , ...,   42.229, -113.5  ,   31.933])
```

When you ask for a preview, Pandas will assign an integer index system.
The index of a Dataframe is a set that consists of a label for each row

| | station_id | longitude | latitude | utc_time | temperature | pressure | humidity | wind_direction |
|---|---|---|---|---|---|---|---|---|
| 0 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 16:00 | -1.7 | 1028.7 | 15 | 215.0 |
| 1 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 17:00 | -3.5 | 1028.4 | 24 | 16.0 |
| 2 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 18:00 | -3.7 | 1028.1 | 27 | 32.0 |
| 3 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 19:00 | -3.9 | 1027.6 | 27 | 21.0 |
| 4 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 20:00 | -4.1 | 1026.7 | 26 | 17.0 |



```
data_meteo.index
```

```
RangeIndex(start=0, stop=158047, step=1)
```

# SETTING AN INDEX - 1) MERGING DATAFRAMES

Quite often, we dont have all the data we need in one place. You can spend a long time trying to merge datasets into one place. Here we can use Pandas to merge to a common index.

In [8]:
```
# Set the index of the meteorological dataframe to be UTC time
Shunyi_met_df.set_index('utc_time')
# What do you see below?
```

Out[8]:

| utc_time | station_id | longitude | latitude | temperature | pressure | humidity | wind_direction | wind_speed | weather |
|---|---|---|---|---|---|---|---|---|---|
| 30/01/2017 16:00 | shunyi_meo | 116.615278 | 40.126667 | -1.7 | 1028.7 | 15 | 215.0 | 1.6 | Sunny/clear |
| 30/01/2017 17:00 | shunyi_meo | 116.615278 | 40.126667 | -3.5 | 1028.4 | 24 | 16.0 | 1.0 | Haze |
| 30/01/2017 18:00 | shunyi_meo | 116.615278 | 40.126667 | -3.7 | 1028.1 | 27 | 32 | | |
| 30/01/2017 19:00 | shunyi_meo | 116.615278 | 40.126667 | -3.9 | 1027.6 | 27 | 21 | | |
| 30/01/2017 20:00 | shunyi_meo | 116.615278 | 40.126667 | -4.1 | 1026.7 | 26 | 17 | | |

```
# Now do the same for the other dataframe
Shunyi_aq_df.set_index('utc_time')
```

| utc_time | stationId | PM2.5 | PM10 | NO2 | CO | O3 | SO2 |
|---|---|---|---|---|---|---|---|
| 01/01/2017 14:00 | shunyi_aq | 386.0 | 477.0 | 116.0 | 6.3 | 4.0 | 8.0 |
| 01/01/2017 15:00 | shunyi_aq | 410.0 | 476.0 | 121.0 | 6.3 | 4.0 | 10.0 |
| 01/01/2017 16:00 | shunyi_aq | 418.0 | 493.0 | 118.0 | 6.8 | 4.0 | 10.0 |
| 01/01/2017 17:00 | shunyi_aq | 474.0 | 532.0 | 133.0 | 7.9 | 5.0 | 11.0 |
| 01/01/2017 18:00 | shunyi_aq | 449.0 | 456.0 | 137.0 | 7.3 | 4.0 | 11.0 |



Merge into one dataframe!



# SETTING AN INDEX - 1) MERGING DATAFRAMES

So each dataframe now uses the same index. We can now combined then into a new dataframe using the:

```
pd.merge(dataframe1,dataframe2,on=<<index name>>)
```

Please check the code snippet below.

```
# Now we can combine these into a new dataframe
#combined_df=Shunyi_met_df.join(Shunyi_aq_df.set_index('utc_time'))

combined_df=pd.merge(Shunyi_met_df,Shunyi_aq_df,on='utc_time')
```

```
# Let's preview the new dataframe. Have the two previous frames merged? What difference can you notice,
# baring in mind that our start times were different? Pandas has sorted this out for us.
combined_df.head()
```

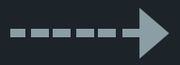
| | station_id | longitude | latitude | utc_time | temperature | pressure | humidity | wind_direction | wind_speed | weather | stationId | PM2.5 | PM10 | NO2 | CO | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 16:00 | -1.7 | 1028.7 | 15 | 215.0 | 1.6 | Sunny/clear | shunyi_aq | 94.0 | 104.0 | 42.0 | 0.2 | 4 |
| 1 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 17:00 | -3.5 | 1028.4 | 24 | 16.0 | 1.0 | Haze | shunyi_aq | 136.0 | NaN | 52.0 | 3.1 | 2 |
| 2 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 18:00 | -3.7 | 1028.1 | 27 | 32.0 | 1.1 | Haze | shunyi_aq | 112.0 | NaN | 43.0 | 2.5 | 3 |
| 3 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 19:00 | -3.9 | 1027.6 | 27 | 21.0 | 0.9 | Haze | shunyi_aq | 110.0 | NaN | 42.0 | 2.7 | 2 |
| 4 | shunyi_meo | 116.615278 | 40.126667 | 30/01/2017 20:00 | -4.1 | 1026.7 | 26 | 17.0 | 1.1 | Haze | shunyi_aq | 112.0 | NaN | 43.0 | 2.9 | N |

BUT...WHAT HAS HAPPENED TO THE INDEX OF OUR NEW DATAFRAME?

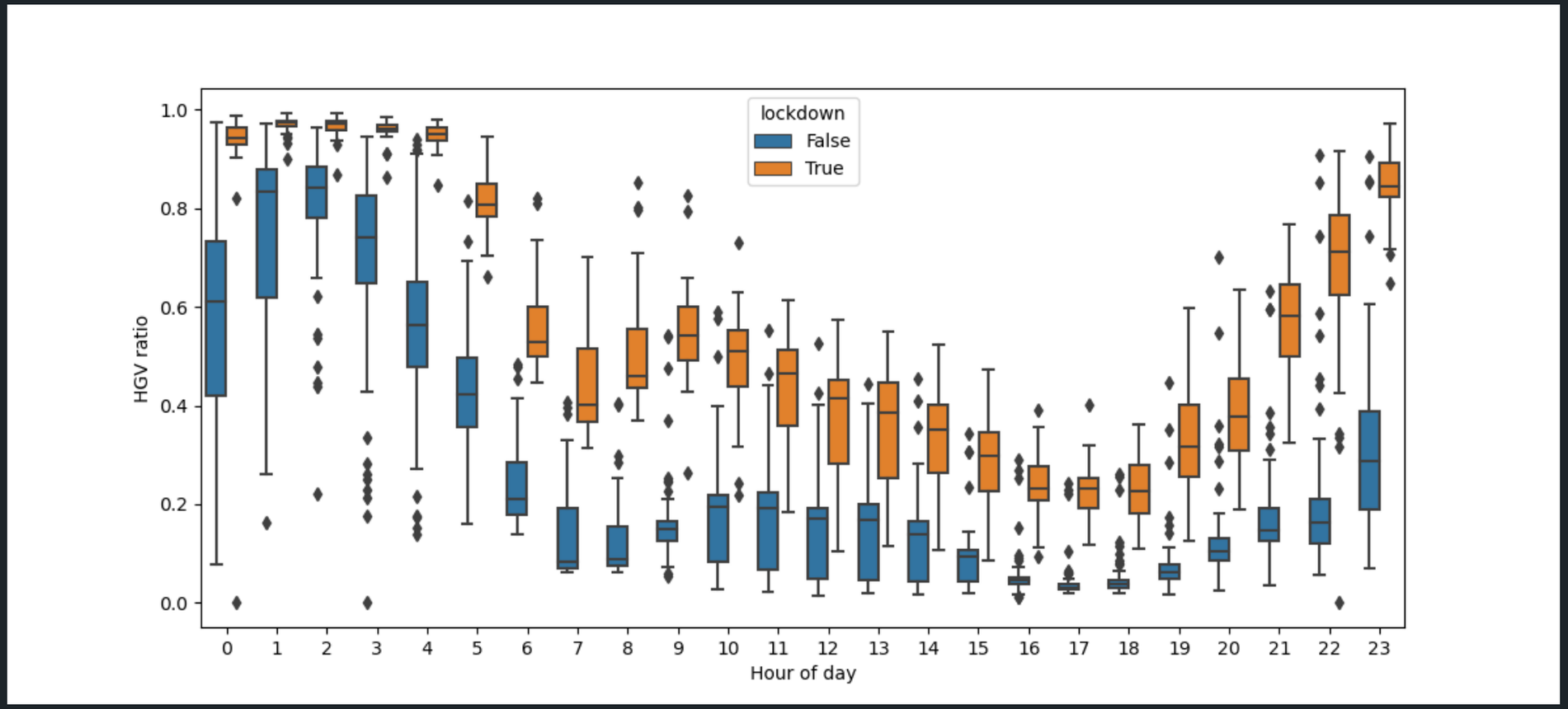
# SETTING AN INDEX - 2) ACCESSING DIURNAL PROFILES

*seaborn: statistical data visualization*



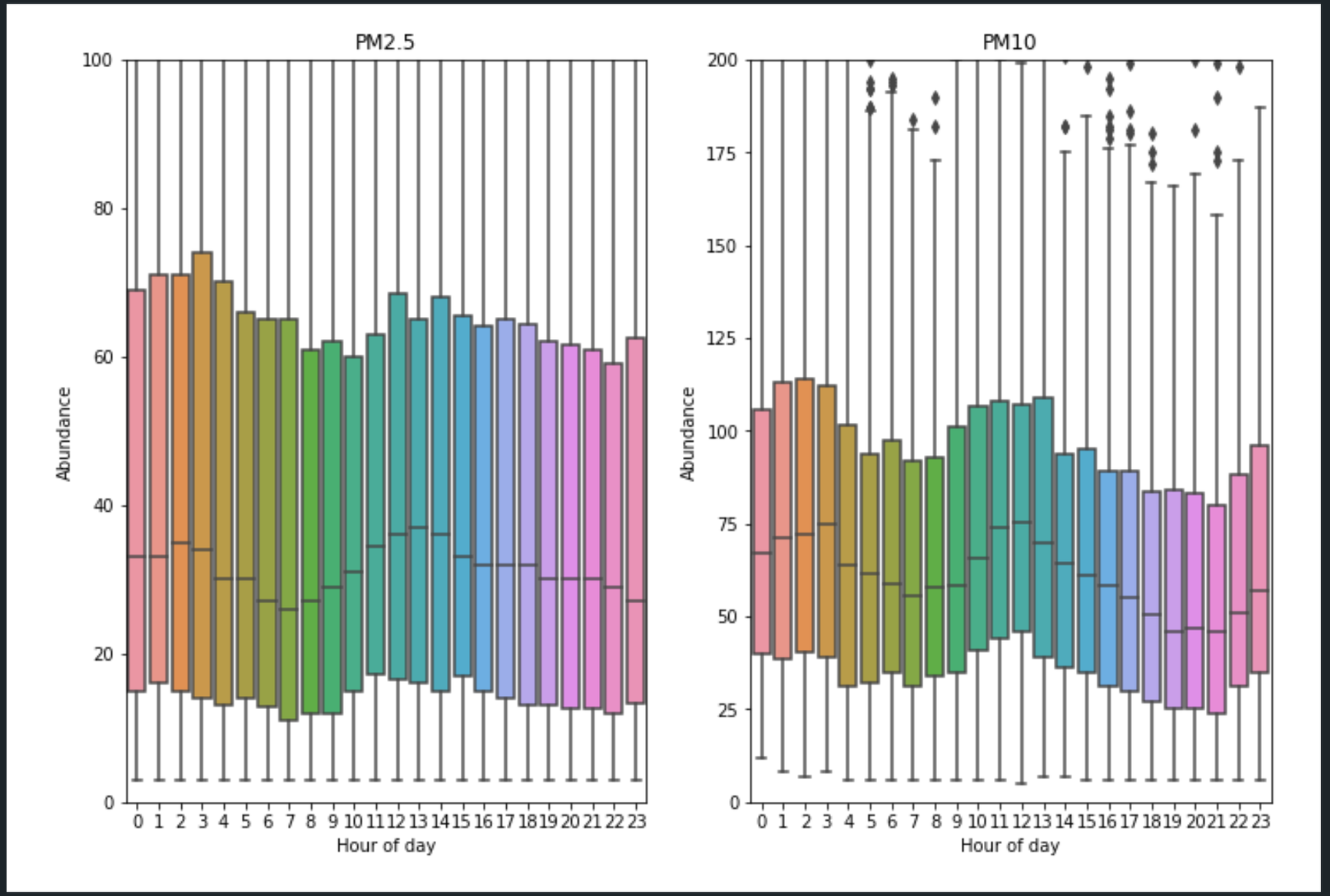





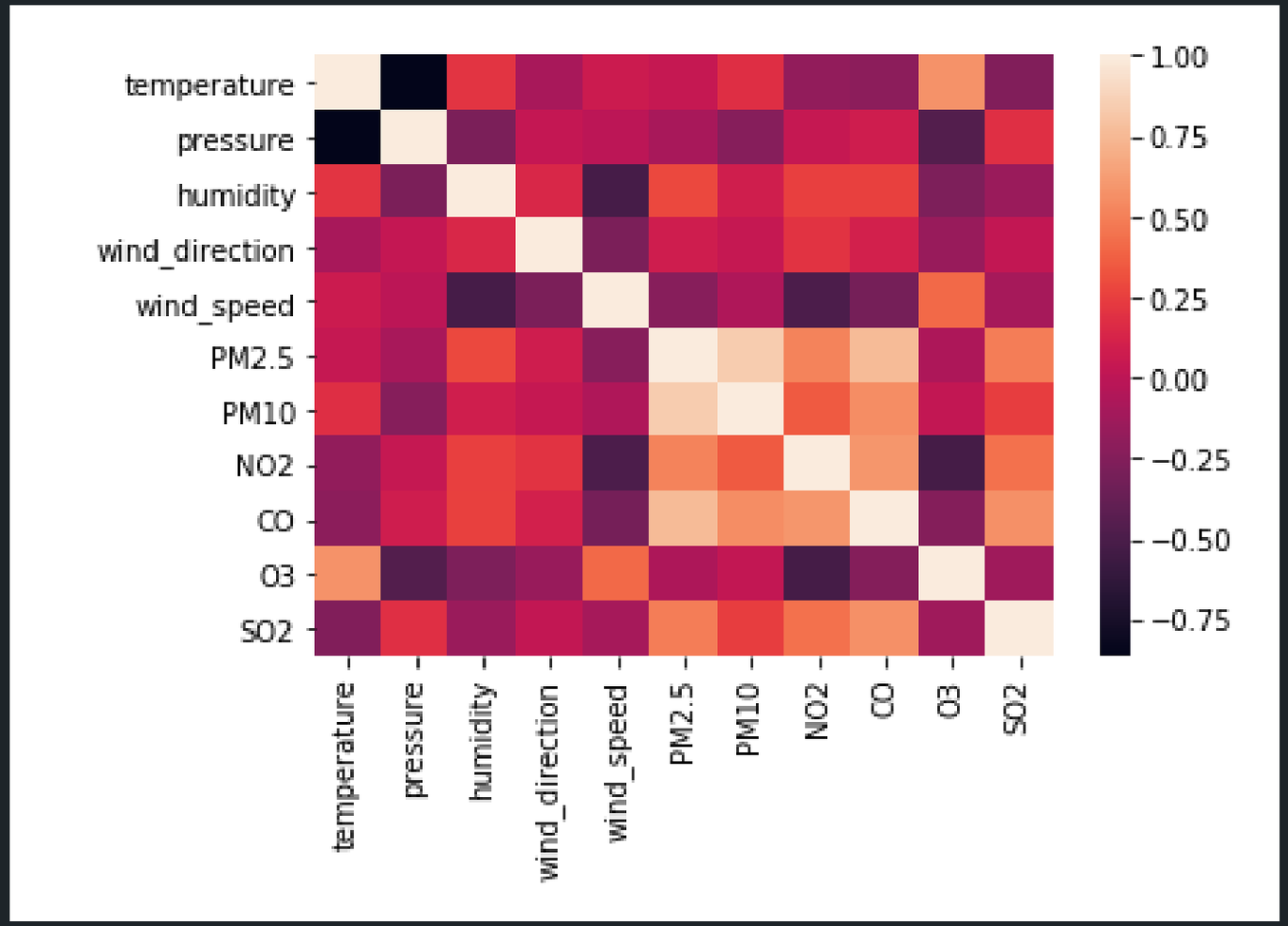



# SETTING AN INDEX - 2) ACCESSING DIURNAL PROFILES

```
x=combined_df.index.hour
y=combined_df['PM2.5']
```

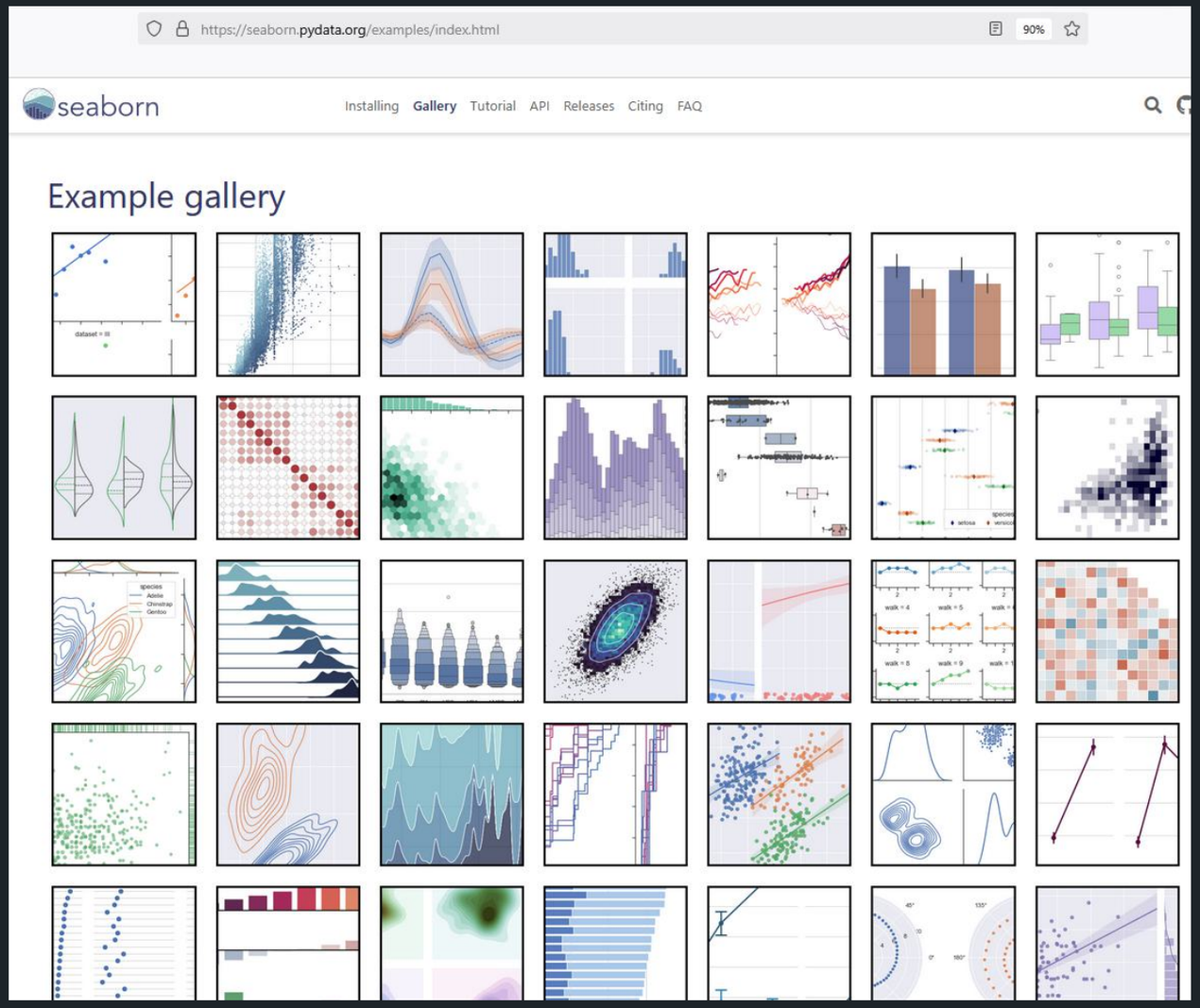


# STATISTICAL ANALYSIS




```
combined_df_new[['temperature','pressure','humidity','wind_direction','wind_speed','PM2.5','PM10','NO2','CO','O3','SO2']].corr()
```

# CHECK OUT SEABORN



# PANDAS AND DATA DISCOVERY

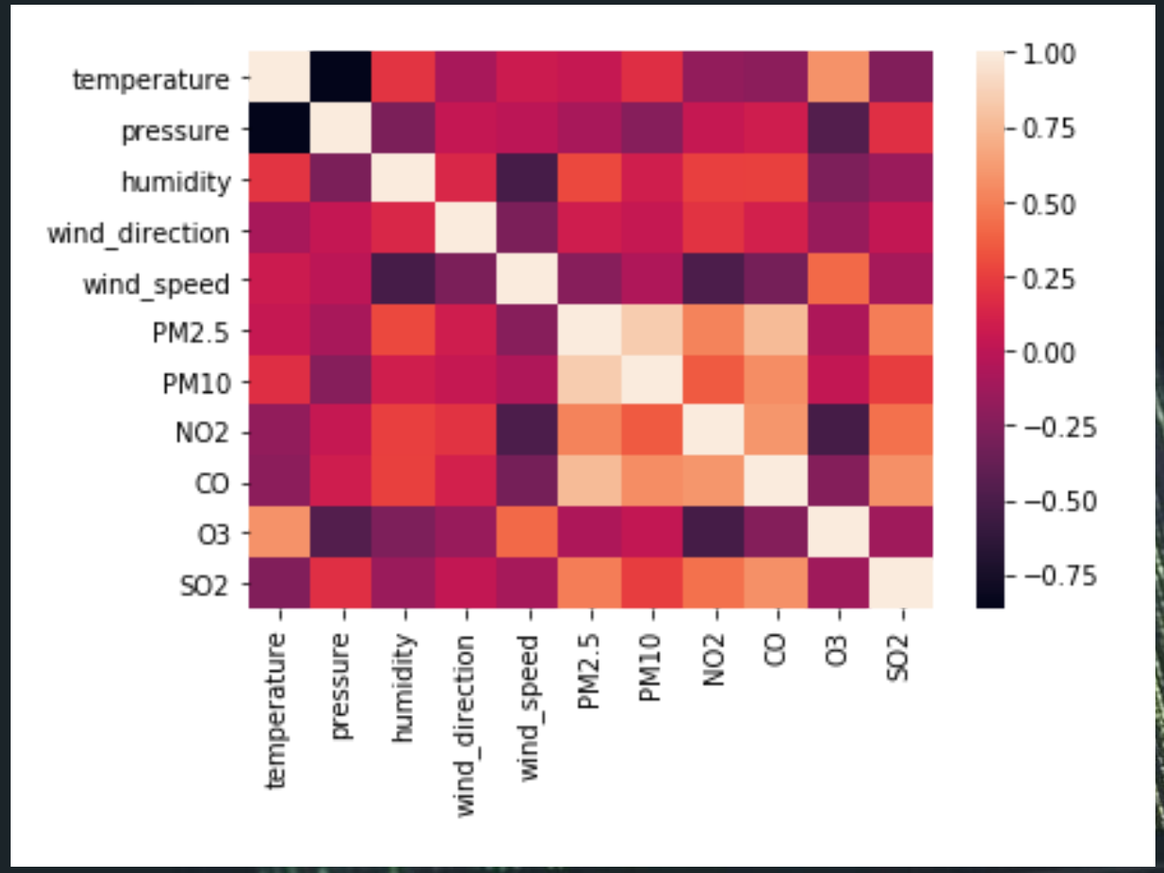


- MANIPULATING DATAFRAMES AND STATISTICAL ANALYSIS
- PRACTICAL 8




David Topping: david.topping@manchester.ac.uk

# LECTURE 7




# SUMMARY

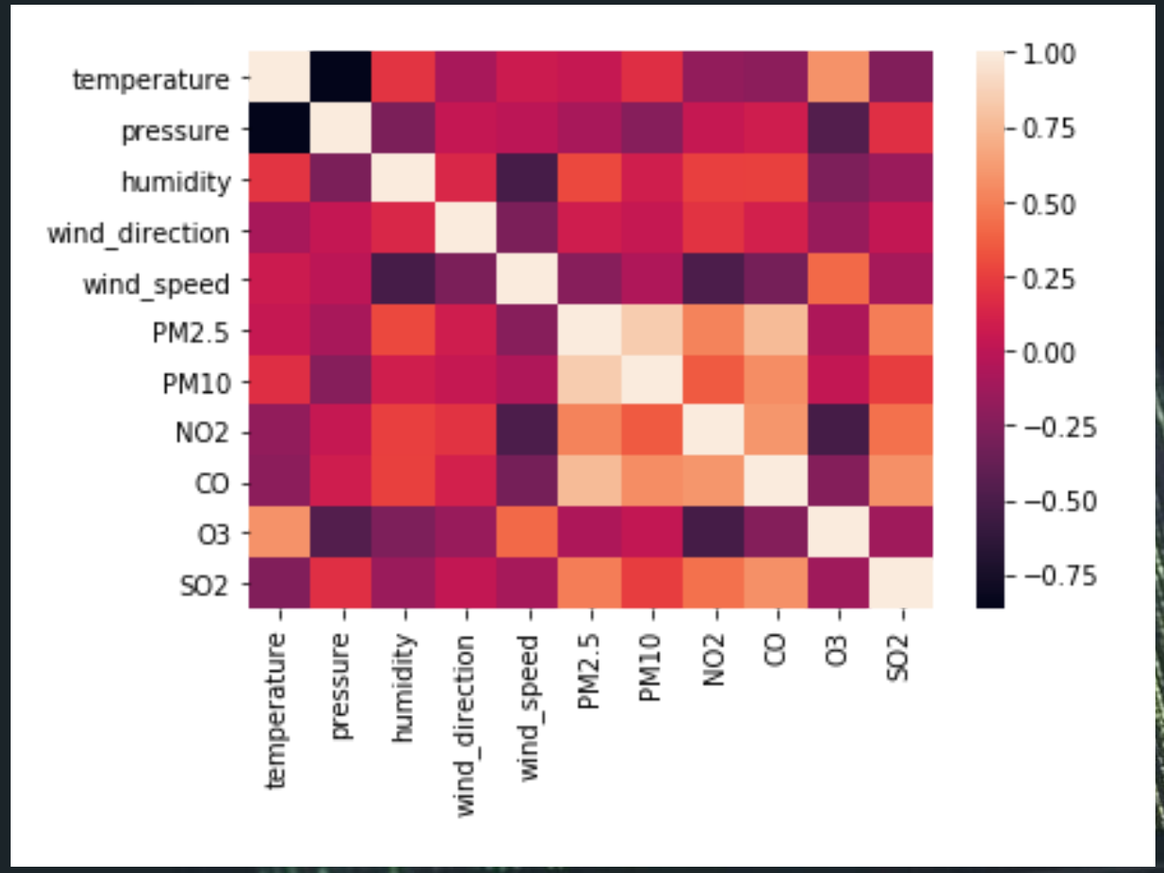
## INTERACTING WITH DATA IN PYTHON

- Data is everywhere.
- Pandas is a powerful module
- We can merge time series data on a common period using simple Pandas functions
- We can extract seasonality/diurnal trends

## STATISTICAL ANALYSIS

- Combined with the Seaborn plotting module, we can visualise multiple statistics describing our data

# PRACTICAL 7




# GOALS

## CODE

- Using Pandas to createg a dataframe
- Merge two dataframes using a common time index
- Perform statistical analysis on dataframes

## GENERAL EXPERIENCE

- Continue your journey with Pandas
- Start to discover relationships in a dataset